Forecasting independent evolution of code clones: a thesis research roadmap

The thesis sits in a **genuine research gap**: while deep learning has saturated clone *detection*, only a single mature line of work (Zhang/Chen/Khoo's CHANN, 2023–2025) has applied deep learning to forecasting clone *evolution*, and even that line predicts *consistent* change rather than *independent* divergence. Forecasting independent evolution is empirically the more important problem—Kanwal, Rattan and Lal (PLOS ONE 2022) found that *the dominant reason clones disappear from systems is unintentional independent evolution of fragments*, not deliberate refactoring. The proposal is therefore well-aimed, but to compete with current literature it needs three upgrades: a hybrid SVN+Git mining pipeline, a richer feature set anchored in process and ownership metrics plus learned embeddings, and an evaluation regime that has become standard in defect-prediction research since 2017 (Tantithamthavorn et al.). This report synthesizes Dr. Mondal's full corpus, the 2018–2026 state of the art, and concrete recommendations for models, features, methodology, and evaluation. PubMed Central

Dr. Mondal's research corpus is the thesis's intellectual scaffold

Manishankar Mondal has produced **roughly 60 clone-related publications** across two theses (MSc 2013, PhD 2017 at Saskatchewan), with five thematic threads the student must position against. His **stability line** (SAC 2012 best paper, EMSE 2018 *Is Cloned Code Really Stable?*) reconciled conflicting earlier studies and showed stability conclusions hinge on metrics, clone type, and granularity. His **SPCP line** introduced Similarity-Preserving Co-change Patterns and the SPCP-Miner tool (SANER 2015), establishing the framework for distinguishing clones that should be refactored from those that should be tracked. His **co-change and evolutionary coupling line**—MSR 2014 (prediction and ranking of co-change candidates), SANER 2020 (HistoRank, association-rule integration), EMSE 2021 (ID-correspondence), ICPC 2021 (FLeCCS), and Science of Computer Programming 2025 (programmer-sensitivity-aware FLeCCS)—is the *direct ancestor* of the thesis topic. The **micro-clone line** (SANER 2018 onward) showed that 1–4 LOC duplicates carry 80% of consistent updates, and the recent ICCIT 2024 paper applies SPCP mining to micro-clones for refactoring identification. Finally, his **bug-propagation line** (ICSME 2015, JSS 2018, ICSME 2017, JSS 2019) demonstrated that Type-3 clones disproportionately propagate bugs and that "context-adaptation bugs" arise precisely when clones evolve independently. Usask + 20

Two of his works function as **must-cite anchors** for this thesis. The MSR 2014 paper, *Prediction and Ranking of Co-change Candidates for Clones*, is the closest precursor methodologically: it predicts which clone fragments should co-change, using association rules over revision history and NiCad over six C/Java systems. The 2020 *Survey on Clone Refactoring and Tracking* (JSS) provides the framing taxonomy and the open-problem list that the thesis can directly target. The student should also cite the canonical book chapter *NiCad: A Modern Clone Detector* (Mondal,

Roy, Cordy, Springer 2021) when justifying tooling choices, and the 2022 JSS paper by Nadim, Mondal, Roy and Schneider that benchmarks clone detectors specifically on co-change-candidate identification—a direct empirical justification for whichever detector the student selects. google + 7

Most importantly, **Mondal's group has not yet applied modern ML/DL to forecasting independent evolution**. The CASCON 2022 paper *Predicting Buggy Code Clones through Machine Learning* (Islam, Paul, Mondal) is the only ML-based predictive work in the corpus. This is the white space the thesis can fill, and doing so as a Khulna University thesis under Mondal's supervision is institutionally well-positioned. Ku

The state of the art and the precise research gap

Outside Mondal's group, three lines define the current frontier. **Zhang, Chen and Khoo's clone-consistency program**—Information and Software Technology 2021 (classical ML on clone-genealogy attributes), Information Sciences 2023 (deep learning with attention; F-measure ≈80%, recall ≈90% across eight projects), and the JCST 2025 hierarchical attention network CHANN (P/R/F ≈ 82%)—is the closest deep-learning work to the thesis. **It predicts consistency, not independence**, and treats clone evolution as a hierarchical sequence (code → context → evolution) processed by a bidirectional circular queue with attention. The thesis can either replicate CHANN as a baseline or invert its target (treat 1 − P(consistent) as the independent-evolution score) for direct comparison. **Wang Hu et al.'s CREC** (TSE 2018) trains tree-based ML on 34 features—including clone history and co-change—to recommend refactoring candidates, achieving 83% within-project F1 and 76% cross-project F1; it is the strongest classical-ML benchmark and shows that history and co-change features dominate code metrics in importance. **Barbour, An, Khomh and Zou's late-propagation prediction work** (Software Quality Journal 2018) trains classifiers on 18 evolution-related features over four long-lived systems (Apache Ant, ArgoUML, jEdit, Maven) using NiCad and iClones, achieving precision 0.91–0.94 and AUC 0.87–0.91. Together with Ehsan, Barbour, Khomh and Zou's chapter in *Code Clone Analysis* (Springer 2021), these establish the methodological template the thesis should emulate. ScienceDirect + 5

Three more recent works should appear in any 2026 literature review. **Assi, Hassan and Zou's** *Unraveling Code Clone Dynamics in Deep Learning Frameworks* (TOSEM 2025) tracks clones across nine DL frameworks over 28 quarters and identifies four longitudinal patterns (Serpentine, Rise-and-Fall, Decreasing, Stable), with bug activity highest in Serpentine—this gives the thesis a fresh empirical motivation and a genealogy-tracking method applicable beyond traditional desktop apps. **Krinke and Ragkhitwetsagul's** *How the Misuse of a Dataset Harmed Semantic Clone Detection* (extended IWSC 2022 best paper, arXiv 2025) is the **most consequential warning paper** of the period: BigCloneBench's semi-automatic labels mean most "true clones" were never validated, and ML models trained on it learn dataset artifacts rather than semantics. The thesis must avoid this trap by either staying grounded in genealogy-based ground truth (preferable) or supplementing with GPTCloneBench (Alam et al., ICSME 2023) and

SemanticCloneBench. **Kanwal, Rattan and Lal's PLOS ONE 2022 study** is the empirical motivation paragraph of the thesis: across five evolving systems, the dominant cause of clone disappearance is independent evolution, not refactoring—directly justifying the prediction target. (arxiv + 3)

The clone-detection state of the art has moved decisively to graph-based and pretrained models: ASTNN (ICSE 2019) and FA-AST GNN (SANER 2020) are mainstream baselines; CodeBERT (EMNLP 2020), GraphCodeBERT (ICLR 2021), and UniXCoder are now standard pretrained backbones; CC2Vec (FSE 2024), SSCD with nearest-neighbor BERT search (SPE 2024), and MAGNET's multi-graph attention network (arXiv 2025, F1 96.5% on BigCloneBench) represent the cutting edge. **For this thesis these matter not as detectors—NiCad remains the appropriate genealogy tool—but as feature generators**: CodeBERT or UniXCoder embeddings of paired clone fragments produce a cosine-distance feature that empirically dominates surface-similarity features like NiCad's similarity score. (Xuwang + 9)

The synthesized gap is sharp: **no published work forecasts which clones in a class will diverge (independent evolution) using a modern hybrid feature set (process + ownership + learned embeddings) on Git-era repositories with rigorous defect-prediction-style evaluation**. That is the contribution slot.

Recommended ML stack for tabular clone-evolution features

For the engineered tabular feature set the student already has, **gradient-boosted decision trees are the empirically established best practice**, not deep learning. Shwartz-Ziv and Armon's *Tabular Data: Deep Learning Is Not All You Need* (Information Fusion 2022), Grinsztajn et al.'s NeurIPS 2022 study explaining *why* tree models still win (irregular targets, uninformative features, non-rotationally-invariant data), and the McElfresh et al. NeurIPS 2023 benchmark all converge on the same conclusion: lightly tuned LightGBM, XGBoost, or CatBoost ties or beats specialized tabular DL across dozens of benchmarks. In software engineering specifically, Tantithamthavorn et al. (TSE 2017) found Random Forest among top defect-prediction classifiers; Ghotra, McIntosh, and Hassan (ICSE 2015) showed RF, Naive Bayes, and Logistic Regression cluster as the top group; Fu and Menzies' FSE 2017 *Easy Over Hard* showed tuned classical ML beats deep learning on defect prediction; Wang/Zhang's IST 2021 clone-consistency study used Bayesian networks as the dominant baseline. (arXiv + 3)

Concretely: build the model portfolio as **LightGBM as primary, CatBoost as secondary** (it natively handles high-cardinality categoricals like `author` and `filepath` without leakage-prone target encoding), **Random Forest and XGBoost as comparators**, **Logistic Regression with L2 as the simple-strong baseline** (mandated by Hall et al. TSE 2012 and Fu/Menzies), and **ZeroR/majority-class as the trivial baseline** mandated by Liu et al.'s MATTER framework (arXiv 2023). An MLP or TabNet/FT-Transformer can be included for completeness but should be expected to tie or lose to LightGBM on this feature shape. **A stacked ensemble of LightGBM + CatBoost + LR typically adds 1–2 percentage points of AUC** and is publication-ready.

Sequence-aware models—LSTMs or Transformers—are appropriate *only* if the student reframes the problem as predicting from a sequence of revision-by-revision feature snapshots; in that case the simplest sensible architecture is a Temporal Convolutional Network or a small Transformer encoder over the snapshot sequence, but this is an optional extension. **Graph Neural Networks are a poor fit** for this tabular data; they are useful only if one builds a clone-genealogy graph or a code-AST graph as input, which is a separate subproject.

Hyperparameter tuning should use **Optuna's TPE sampler with 50–100 trials inside a nested cross-validation loop** (Akiba et al. KDD 2019; Tantithamthavorn TSE 2018 showed tuning yields ≈0.4 AUC and reorders feature-importance rankings). Tu and Nair (FSE 2018) caution that no single optimizer dominates, so reporting tuning budget honestly matters. For class imbalance (independent-evolution events are typically a minority class), **first try** `scale_pos_weight` / **class-weighted loss** rather than SMOTE—Tantithamthavorn et al.'s TSE 2019 study showed rebalancing improves recall but hurts precision and AUC and *changes which features the model considers important*. If SMOTE is needed, follow Agrawal and Menzies' ICSE 2018 *SMOTUNED* recipe (tuned SMOTE inside CV folds, never before splitting) since they showed tuned SMOTE adds ~60% AUC over default SMOTE. PubMed Central + 3

Feature engineering: process and ownership signals are the single biggest opportunity

The current feature set is product-metric heavy and process-metric light. Rahman and Devanbu's ICSE 2013 paper *How, and why, process metrics are better than code metrics* and its 2022 large-scale replication by Majumder et al. (EMSE) are the strongest reasons to expand. **Five additions will likely deliver the largest predictive gain**: ACM Digital Library

First, **ownership and authorship divergence features** following Bird et al.'s FSE 2011 *Don't touch my code!*: a binary `same_author` flag for the two clone fragments, the major-author proportion (TCO), the count of minor authors (TCO < 5%), and a recency-of-contribution variable. Bird's central finding—that fragments with many minor authors fail more often—maps directly to the hypothesis that fragments with non-overlapping contributor sets evolve independently. This is the cheapest and most under-exploited family of features. Second, **process metrics**: per-fragment churn (lines added/deleted/modified over a rolling window), number of past changes (NoC), number of past bug-fixes (NoB), file age, and number of distinct authors (Moser et al. ICSE 2008; Nagappan and Ball ICSE 2005). Third, **a CodeBERT (or UniXCoder) cosine distance** between the two fragments. Empirically this captures semantic similarity that NiCad's surface similarity misses and is the single most informative learned feature in 2024–2026 clone studies. Fourth, **AST tree-edit distance via APTED or GumTree** between paired fragments, which converts NiCad's coarse similarity score into a principled structural distance and unlocks change-kind features (rename, statement-insert, condition-modify) when applied to historical diffs. Fifth, **survival/time-to-event encoding**: censoring flag plus duration to next change, enabling Cox regression or DeepHit as a parallel modeling track—

this reframes the problem as "*when* will independent change happen?" rather than just "*will* it?", which is novel in clone-evolution research. Semantic Scholar

Secondary additions worth including: McCabe cyclomatic complexity and Halstead Volume per fragment plus their pairwise *deltas* (the delta is the active signal); for class/method granularity, the CK metrics (CBO, RFC, WMC, DIT, NOC, LCOM) and a CK-distance between hosts; clone-group-level features such as group size (beyond pairwise), the SPCP-history fraction (proportion of past changes that were similarity-preserving co-changes—a direct prior on future co-change behavior), and the dispersion entropy of group members across packages (Mondal et al.'s "dispersion of changes" metric, Sci. Comp. Prog. 2014). Token-level Shannon entropy of identifiers and Hindle et al.'s *naturalness* perplexity offer cheap signals of cognitive load that correlates with divergent maintenance. Springer google

Before training, run **AutoSpearman feature reduction** (Jiarpakdee, Tantithamthavorn, Hassan TSE 2019): drop one of any pair of features with Spearman $\rho > 0.7$ plus a VIF check. The current set has obvious correlations (`co_change_count` with `late_propagation_count` and `change_proneness`; `nlines` with `class_size`) that will distort interpretation if left untreated.

Methodology critique and concrete improvements

The proposal's six-step pipeline (SVN snapshot mining → NiCad → diff → genealogy → evolutionary analysis → ML) is a sound foundation but has **fourteen gaps that should be closed before defense**.

The most consequential is **mining only SourceForge SVN in 2026**. GitHub officially sunset Subversion bridge support on January 8, 2024; SourceForge has had documented SVN data-loss incidents (notably April 2018); and most "classic" subject systems (jEdit, Apache Ant, JabRef) migrated to Git years ago, so SVN history is stale. The right answer is a **hybrid pipeline**: reproduce Mondal's canonical SVN baselines for direct comparability with prior literature, then *extend* the same projects' Git histories using PyDriller or Perceval, and add 2–3 born-on-Git systems (Apache Commons, Elasticsearch, modern JabRef on GitHub). This single change converts an external-validity threat into a contribution. The standard subject-system list to anchor against is the canonical **Mondal six**: Ctags, Carol, FreeCol, jEdit, JabRef, MonoOSC, optionally augmented with ApacheAnt, ArgoUML, and Camellia. Aim for **500–2,000 revisions per system, ≈10K commits in total**—matching Mondal's typical scale and Saha et al.'s gCad evaluation (MSR 2013). SourceForge Hacker News

A single clone detector introduces tool bias. **Keep NiCad as primary** (matches the Khulna pipeline and Mondal's prior work) but also run **iClones**, which was purpose-built for incremental cross-revision tracking and avoids costly N×N pairwise mappings (Göde and Koschke, CSMR 2009; JSEP 2013). Cross-validate a sample with DECKARD or CCAligner. **Use gCad** (Saha, Roy, Schneider, ICSM 2013) for genealogy reconstruction rather than a custom implementation—it is the validated lineage-tracking framework, supports six genealogy classifications including

"consistently changed" and "inconsistently changed," and saves months of engineering. Replace plain line-diff with **AST diff (GumTree or ChangeDistiller)** so that change kinds become categorical features for ML. **Operationalize "independent evolution" using Mondal's per-commit SPCO/non-SPCO definition** (a commit modifies one fragment without modifying its sibling within the same commit) as primary, and report per-revision-pair as sensitivity analysis. Semantic Scholar + 2

For validation, plan a **stratified random sample of ≈384 clone pairs/genealogies** (95% confidence, 5% margin—the standard Krinke/Ragkhitwetsagul 2025 used 406), inspected by **two independent raters** with **Cohen's κ reported** (target >0.7). Use the SZZ algorithm (Sliwerski-Zimmermann-Zeller) for bug-link traceability rather than keyword search. Threats to validity should be framed using the Wohlin/Runeson template: construct (clone-detector reliability, definition of independence), internal (feature leakage from future commits, author-ID confounds), external (subject-system selection, language coverage), and conclusion (statistical-test choice, multiple-comparison correction). Release a **Zenodo-archived replication package** with NiCad configurations, genealogy XMLs, feature CSVs, trained models, and evaluation scripts—targeting ACM Artifact Review badges (Available, Functional, Reusable). arxiv

Six high-impact extensions can elevate the thesis from competent to excellent: (1) cross-project leave-one-out evaluation for transferability claims; (2) survival-analysis formulation with Cox or DeepHit for time-to-event prediction; (3) a Type-4/semantic clone extension using CodeBERT or GraphCodeBERT against GPTCloneBench/SemanticCloneBench; (4) an industrial case study with a Bangladeshi software firm—a hallmark of strong empirical theses; (5) SHAP-derived actionable developer guidelines (e.g., "when CK-distance > X, last-author mismatch, and CodeBERT cosine < Y, expect independent evolution within 90 days"); and (6) a tooling deliverable—a VS Code or IntelliJ plugin that surfaces high-divergence-risk clones, mirroring Teamscale's industrial direction and HAnS's SPLC 2024 plugin. Teamscale Chalmers University of Tec…

Evaluation: the modern defect-prediction template applies directly

The thesis should follow the evaluation regime now standard in the defect-prediction literature post-Tantithamthavorn (TSE 2017–2019). **Replace F1 as the primary metric with the Matthews Correlation Coefficient (MCC)**: Yao and Shepperd's PROMISE 2020 study showed F1 and MCC disagreed on direction in 23% of paired comparisons, and Chicco and Jurman's papers argue MCC is the only single metric that scores high *only* when all four confusion-matrix cells are good. The recommended primary panel is **MCC, AUC-ROC, and PR-AUC** (PR-AUC matters under heavy class imbalance per Saito and Rehmsmeier 2015). Report Precision, Recall, F1, balanced accuracy, G-mean, and Brier score as secondary metrics for comparability with prior clone-evolution literature. If reframed as an inspection-prioritization task, add an effort-aware metric (Popt or PofB20, Mende and Koschke 2009; Kamei et al. TSE 2013).

For cross-validation, use **stratified out-of-sample bootstrap with ~100 repetitions** (Tantithamthavorn TSE 2017) for snapshot data, **walk-forward validation respecting commit/release order** if features encode temporal information (Falessi et al. 2018; Bangash et al. EMSE 2020), and add a cross-project (leave-one-project-out) experiment. Always stratify on the target and group-split on `clone_group_id`/`filepath` to prevent leakage from sibling rows. Mandatory baselines per Liu et al.'s MATTER framework: ZeroR, stratified random, single-feature predictors (e.g., predict by `co_change_count` alone), an unsupervised LOC-based predictor, and tuned Logistic Regression. Beyond these, compare against **CREC's tree-based ML** (Wang Hu, TSE 2018), **Mondal's association-rule SPCP-Miner** (SANER 2015), **Barbour et al.'s late-propagation classifier** (SQJ 2018), and—critically—**a CHANN-style hierarchical attention network** (JCST 2025) trained to predict 1 − P(consistent) on the same data.

Statistical analysis should use **Wilcoxon signed-rank for paired comparisons, Cliff's δ for effect size, and the non-parametric Scott-Knott ESD test (NPSK v2.x)** for ranking many classifiers into statistically distinct groups—the de facto SE standard since Tantithamthavorn TSE 2017. Apply Holm correction for multiple comparisons. For interpretability, use **TreeSHAP** (Lundberg 2018; Lundberg et al. *Nature Machine Intelligence* 2020) for global summary plots and local force/waterfall plots on representative high/medium/low-risk clones, with permutation importance as an independent check; LIME can be added but Shin et al. (TSE 2023) warn that LIME explanations are unstable across resampling settings. Jiarpakdee et al.'s TSE 2020 work on model-agnostic explanations and Tantithamthavorn and Jiarpakdee's *Explainable AI for Software Engineering* tutorial (xai4se.github.io) are the canonical references. arxiv

Recent works (2023–2026) the proposal must cite

The proposal cites only up to ~2022, and the literature review must be refreshed with at least these eleven post-2022 entries: **Assi, Hassan and Zou** (TOSEM 2025) on clone dynamics in DL frameworks; **Krinke and Ragkhitwetsagul** (arXiv 2025) on BigCloneBench misuse; **Zhang/Chen/Khoo et al.** (Information Sciences 2023; JCST 2025 CHANN) on clone-consistency deep learning; **Alam et al.** (ICSME 2023) on GPTCloneBench; **Nadim et al.** (ICSME 2024) on the generalization failure of DL clone detectors across BigCloneBench/SemanticCloneBench/GPTCloneBench; **Wu et al.** (FSE 2024) on CC2Vec contrastive embeddings; **Chochlov et al.** (SPE 2024; arXiv 2025) on LLM-ensemble clone detection; **Kaur and Rattan** (Computer Science Review 2023) and **Quradaa et al.** (PLOS ONE 2024) systematic reviews on ML in clone research; **Kanwal, Rattan and Lal** (Computer Science Review 2025) SLR on clone refactoring; **Mondal's own ICCIT 2024 paper** with Elahi on identifying refactorable micro-clones through SPCP mining; and **Mondal's MSR 2025 mining-challenge paper** with Shanto et al. on dependent vs independent artifacts in the Maven ecosystem—a direct conceptual neighbor. ku Ku

Conclusion: the thesis is one disciplined upgrade away from a publishable contribution

The thesis topic is timely, supervisor-aligned, and genuinely novel: independent evolution prediction with modern features and rigorous evaluation has not been done, despite Kanwal et al.'s empirical demonstration that independent evolution drives most clone disappearance. The single highest-leverage change is **upgrading the data pipeline from SVN-only SourceForge to a hybrid SVN+Git regime** that includes the post-migration histories of Mondal's canonical subject systems plus 2–3 born-on-Git projects—this dissolves the most likely external-validity attack and brings the work into 2026 norms. The second is **reframing feature engineering around process metrics, ownership divergence (Bird et al.), and CodeBERT/UniXCoder embedding distance**, which the empirical defect- and change-prediction literature shows dominate static metrics. The third is **adopting the post-2017 defect-prediction evaluation template**: MCC + AUC + PR-AUC, walk-forward CV, Scott-Knott ESD ranking, AutoSpearman feature pre-screening, TreeSHAP interpretability, and Tantithamthavorn-style nested-CV hyperparameter tuning with Optuna.

If the student delivers LightGBM/CatBoost as primary models with this feature set, benchmarked against CREC, SPCP-Miner, Barbour's late-propagation classifier, and a CHANN-equivalent deep baseline, on six to eight subject systems with manual κ-validated ground truth and a public replication package, the resulting work is **directly publishable at IWSC, SANER, ICSME, or in the *Journal of Systems and Software***. The optional extensions—survival-analysis time-to-divergence modeling, an industrial Bangladeshi case study, and an IDE plugin deliverable—each add a clear venue path (EMSE, ICSE-SEIP, ASE tool track) and convert the thesis from an empirical study into a coherent program of work that aligns naturally with Dr. Mondal's existing research trajectory.